

În general, senzorii sunt rezistențe variabile care își modifică valoarea în funcție de factorii de mediu. Astfel, căderea de tensiune de pe senzor se schimbă.

Pinii analogici funcționează ca un voltmetru care măsoară căderea de tensiune de pe senzor și o transformă într-o valoare finită. Dispozitivul care face conversia din voltaj într-o valoare întreagă, transmisă plăcii, se numește **convertor analog-digital** (ADC). Valorile întregi returnate depind de numărul de biți integrați în ADC. Dacă considerăm n numărul de biți al ADC-ului, valorile sunt în intervalul $0-2^{n-1}$. De exemplu, convertorul analog-digital integrat în plăcuțele Arduino are 10 biți, deci valorile citite de pe pinii analogici sunt în intervalul $0-1023$.

Putem astfel să conectăm un fotorezistor la un pin analog, iar în funcție de luminozitatea mediului vom citi valori mai mici sau mai mari. Dacă identificăm o luminozitate prea mică, putem scrie valoarea 1 pe un pin GPIO conectat la un LED și să aprindem LED-ul, obținând un prototip al unui sistem de iluminare inteligent.

Am stabilit că pentru a citi semnale analogice vom folosi pini conectați la un ADC, dar ce facem pentru a trimite semnale analogice mediului?

În primul rând, există posibilitatea de a folosi un convertor digital-analogic (DAC). El transformă o valoare digitală într-un semnal analogic. Putem deci să scriem o valoare pe un astfel de pin pentru a controla tensiunea de ieșire a pinului. Astfel de pini sunt folosiți pentru a controla motoare.

Pe de altă parte, convertoarele de la digital la analogic sunt destul de scumpe, de aceea multe dispozitive încorporate nu expun astfel de pini. În schimb, ele folosesc o metodă mai accesibilă de simulare a semnalelor analogice, PWM.

Pulse Width Modulation (PWM) este o modalitate de a simula semnale analogice prin mijloace digitale. Putem realiza acest lucru prin alternarea rapidă între semnale care pot avea doar două valori (0/1), dar prin modificarea factorului de umplere, adică timpul în care pinul are valoarea 1. De multe ori, receptorul semnalului va face o medie a celor două valori alternative.

De exemplu, dacă avem un LED care clipește foarte repede, ochiul uman îl va percepe ca un LED ce luminează la o anumită intensitate. În funcție de raportul între perioada în care LED-ul este aprins versus perioada de timp în care el este stins, lumina va fi percepută la o intensitate mai mare sau mai joasă.

Acești pini pot fi folosiți pentru a controla LED-uri sau chiar motoare.

15.3 Magistrale de comunicare

Până acum am enumerat modalitățile de conectare a perifericelor simple, folosind pini digitali sau analogici. Cum am menționat deja, multe din perifericele existente sunt sisteme complexe, care au un procesor integrat și care trimit date structurate. În plus, există multe periferice care comunică wireless cu dispozitivele încorporate.

Pentru a interacționa cu aceste dispozitive de intrare/ieșire complexe, există diverse protocoale prin fir (ex. serial, SPI (*Serial Peripheral Interface*), I²C, Modbus, CAN (*Controller Area Network*)) sau wireless (ex. Bluetooth, Wi-Fi, LoRa, Z-Wave). Unele din aceste protocoale sunt valabile și pentru calculatoarele clasice. De exemplu,

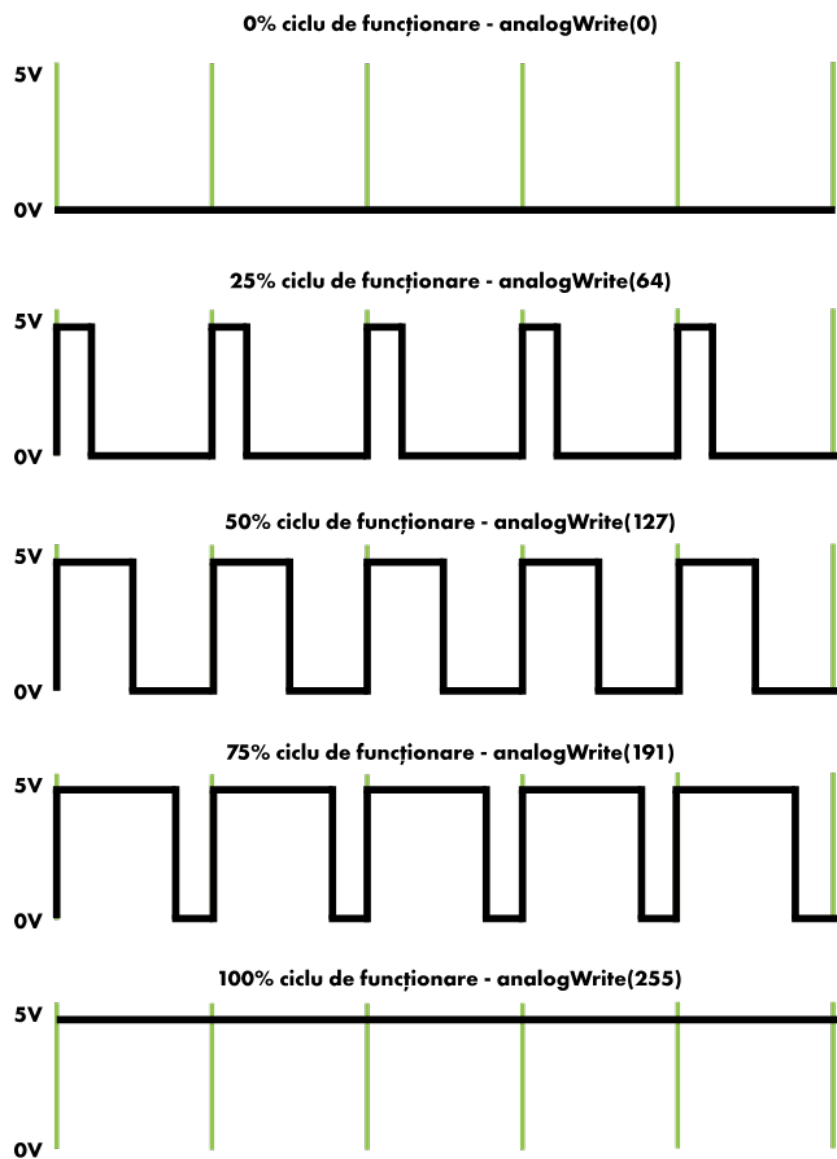


Figura 15.3: Pulse Width Modulation (PWM)

porturile USB implementează o conexiune serială. De asemenea, utilizăm zilnic mouse-uri wireless, unele folosind Bluetooth pentru comunicare. Pe de altă parte, multe din protocoalele pe care le vom descrie mai departe au fost implementate special pentru comunicarea între dispozitivele încorporate și periferice.

15.3.1 Comunicare prin fir

Unele din primele periferice complexe au fost dezvoltate pentru a fi conectate direct la calculatoare, transmitia făcându-se prin intermediul firelor. Cea mai cunoscută astfel de conexiune este conexiunea serială.

În continuare vom descrie câteva din protocoalele pe care mulți senzori și alte periferice le implementează.

15.3.1.1 Comunicarea serială

Comunicarea serială (Figura 15.4) e unul dintre cele mai cunoscute și simple protocoale. Protocolul presupune trimiterea datelor între dispozitive bit cu bit, pe rând. În contrast, există linii de comunicație paralele, unde datele sunt trimise în paralel. Avantajul comunicării seriale este simplitatea.

Pentru a implementa un protocol serial sunt necesare minim două linii de comunicație, RX și TX. Pe fiecare linie se trimit date într-o direcție. Fiecare dispozitiv trimite date pe TX și primește date pe linia RX. Astfel, când conectăm două dispozitive, linia RX a unuia se va conecta la linia TX a celuilalt și invers.

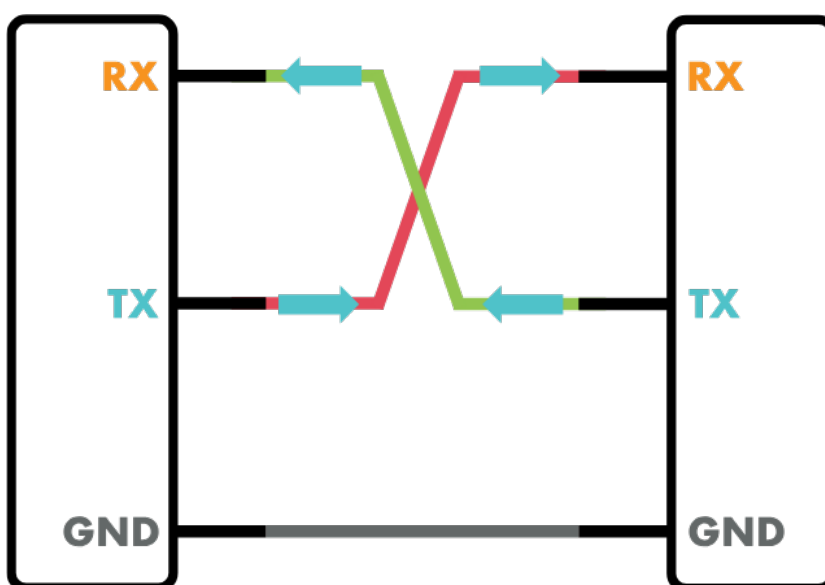


Figura 15.4: Conexiune serială

Pentru a se asigura un transfer corect de date, protocolul serial se bazează pe următoarele elemente:

- biți de date - datele efective transmise între dispozitive;

- biți de sincronizare - sunt biți transmiși înainte și după biții de date, pentru a marca începutul și sfârșitul blocului de date;
- biți de paritate - e o modalitate simplă de a face verificarea datelor transmise (o sumă de control foarte simplă); dacă trimitem 8 biți de date și din acei 8 biți 5 conțin valoarea 1, datele trimise conțin un număr impar de valori de 1, deci bitul de paritate va fi setat la 1, dacă numărul de valori de 1 este par, bitul de paritate va fi setat la 0;
- baud rate - specifică viteza de transmitere a datelor; este foarte important ca ambele dispozitive conectate să transmită date la aceeași viteză.

Protocolul serial poate fi folosit doar la comunicarea între două dispozitive. Asta înseamnă că fiecare dispozitiv periferic conectat are nevoie de un port serial separat, făcând conectarea multor dispozitive nepractică.

15.3.1.2 SPI

SPI e un protocol mai complex, care necesită 3 linii pentru comunicare. Datele sunt transmise pe liniile MISO (*Master In Slave Out*) și MOSI (*Master Out Slave In*). Cea de-a treia linie este folosită pentru sincronizare (SCLK).

SPI vine ca o îmbunătățire a conexiunii seriale, încercând să asigure sincronizarea celor două dispozitive care comunică, transmisia de date realizându-se într-un mod mai eficient prin reducerea erorilor rezultate de la transmisie. Astfel, SPI se folosește de o a treia linie de comunicare dedicată sincronizării transmisiei de date.

Un alt avantaj al SPI e că are suport pentru mai multe periferice, denumite *slave*, conectate la un dispozitiv principal, numit *master*. Fiecare dispozitiv este activat sau dezactivat la un moment dat folosind un pin GPIO oarecare setat la valoare 0 sau 1.

15.3.1.3 I²C

I²C, sau IIC (*Inter-Integrated Circuit*), e un alt protocol care necesită două linii pentru transmiterea de date, SDA și SCL. SDA e linia pe care se transmit efectiv datele, în timp ce SCL e linia de sincronizare.

Avantajul protocolului I²C este că putem conecta mai multe dispozitive simultan. Avem dispozitive master și slave. În cazul I²C este foarte importantă sincronizarea dintre dispozitive. Avem o singură linie pe care se trimit toate datele. De aceea, comunicarea este mereu inițiată de un master care cere informații de la unul din dispozitivele slave conectate.

Fiecare dispozitiv are o adresă formată din 7 biți, adresă înregistrată în periferic. Sistemul master interoghează pe rând fiecare adresă, putând astfel să descopere adresele tuturor dispozitivelor legate la sistem.

15.3.1.4 Alte protocoale folosite industrial

15.3.1.4.1 Modbus Un protocol folosit destul de mult în industrie este ModBus. Spre deosebire de cele enumerate anterior, acesta este un protocol software. El definește cum arată datele transmise între dispozitive, fără a defini însă o infrastructură de comunicație. Există diferite variante, în funcție de ce mijloc de transport de date se folosește. Exemple sunt o legătură serială sau Ethernet (rețea). Similar cu I²C, fiecare periferic conectat la Modbus are o adresă unică.

15.3.1.4.2 CAN Un protocol folosit în industria Auto este CAN (*Car Area Network*). Acesta leagă sistemele de control din autoturisme de periferice. Aproape toate sistemele electronice din autoturisme sunt legate la magistrala CAN. Fiecare dispozitiv conectat la magistrală se numește nod. Fiecare nod are un ID (adresă) unic. Protocolul are o variantă de viteză redusă, tolerantă la erori. Protocolul a fost gândit pentru transmisie în timp real.

15.3.2 Comunicarea wireless

Pe măsură ce platformele IoT au evoluat, ele au fost incluse în multe sisteme întinse pe arii largi, cum ar fi sistemele agricole. Pentru astfel de cazuri, conectarea fizică a senzorilor la sistemele de procesare este costisitoare și consumă mult timp. Drept urmare, au fost implementate o mulțime de microcontrolere și calculatoare integrate care suportă conexiuni WiFi. Cu toate acestea, deoarece multe dintre dispozitivele utilizate în sistemele inteligente funcționează pe baterie, a existat necesitatea unui protocol care să implice consum redus de energie. Astfel, au apărut mai multe protocoale wireless, special concepute pentru IoT.

15.3.2.1 Bluetooth

Acesta a fost unul dintre primele protocoale implementate în sistemele IoT. Pe măsură ce domeniul s-a dezvoltat și Bluetooth a rămas principalul protocol de comunicare între dispozitive, acesta a fost extins, implementându-se Bluetooth 4.0, sau *Bluetooth Low Energy* (BLE). Acesta este special conceput pentru a consuma puțină energie în procesul de transfer al datelor.

15.3.2.2 Z-wave

Este un protocol utilizat în principal pentru sistemele de automatizare a caselor (întrerupătoare, termostate, încuieri inteligente). Comunicația între dispozitive se face prin undă radio, iar dispozitivele trebuie să fie amplasate la o distanță de maxim 100 m.

15.3.2.3 Sigfox

Este un protocol conceput pentru transmiterea de date între dispozitive situate la distanțe considerabile unul de celălalt. Principalul avantaj al Sigfox este că poate fi utilizat în

agricultură sau în alte domenii în care dispozitivele trebuie să transmită date pe distanțe mari. Folosind Sigfox, putem transmite date pe distanțe de până la 50 km în câmp deschis.

15.3.2.4 LoRa

Numele protocolului provine de la sintagma Long Range, rază lungă. Este similar cu Sigfox, permițând schimbul de pachete pe distanțe lungi, până la 50 km în câmp deschis, folosind foarte puțină energie pentru transmiterea de date.

15.3.3 Sisteme gateway

Pentru a permite acestor periferice să trimită și să primească date de la dispozitivele integrate, cele două părți trebuie să implementeze același protocol. Prin urmare, construirea unei soluții IoT care este, de exemplu, compatibilă atât cu periferice LoRa, cât și cu periferice Sigfox, necesită implementarea celor două protocoale în logica aplicației, iar introducerea unui al treilea protocol necesită alt efort. De aceea, pentru comunicarea cu astfel de periferice se folosesc, de obicei, gateway-uri. În general, gateway-urile sunt dispozitivele care implementează protocolul de comunicație periferic, pe de o parte, și un protocol TCP / IP standard, pe de altă parte, pentru a comunica cu dispozitivul încorporat. Rezultatul este că aplicația care rulează pe dispozitivul încorporat comunică doar cu gateway-urile, folosind un singur protocol, și este independentă de senzorii conectați la sistem.

15.4 Dezvoltarea programelor pentru sisteme integrate

În procesul de dezvoltare al aplicațiilor pentru sisteme integrate trebuie să ținem cont de faptul că aplicația dezvoltată va funcționa pe o perioadă îndelungată și pe un sistem autonom. Astfel, trebuie să ne asigurăm că orice posibile erori sunt tratate cu atenție, fără a necesita intervenție umană.

Putem identifica următorii pași în dezvoltarea programelor pentru aceste sisteme:

1. scrierea programului
2. testarea pe mai multe dispozitive
3. instalarea aplicației pe toate dispozitivele necesare
4. aducerea la zi a aplicației

În ceea ce privește scrierea programului, procesul nu este diferit de cel pentru aplicațiile obișnuite. În schimb procesul de rulare a aplicației pe dispozitiv este diferit. Având în vedere că ne referim la dispozitive care nu au fost concepute pentru interacțiunea directă cu utilizatorul, acestea nu au, în general, interfață grafică. Practic, noi vom scrie programul folosind un calculator obișnuit, după care vom încărca executabilul pe dispozitivul integrat.